Response Letter

We thank the reviewers for their valuable comments. The paper has been carefully revised according to the meta-review, and all of the requested changes were implemented. Below is our detailed response to the meta-review and each reviewer.

Replies to Requested Changes

Requested Change 1: Report a full evaluation of the native test seeds baseline in the main evaluation section vs. Discussion

Reply: Thanks for the suggestion. After 24 hours, the fuzzers that used the combination of SEDAR's seeds and native test cases covered 10.77%-28.41%, 12.53%-16.20%, and 4.90%-9.73% more branches, and found 13, 5, and 2 more bugs than only utilizing native test cases in MonetDB, DuckDB, and ClickHouse, respectively. We have reported the full evaluation of introducing native test cases into initial seeds in Section 5.3, and have removed the old paragraph from the Discussion section.

Specifically, we added two sets of initial seeds: one is using the native test cases only (with mutability refinement enabled, labeled as SEDAR-GRIFFIN^{N-} and SEDAR-SQUIRREL^{N-}), and the other one is the combination of native test cases and the transferred seeds (labeled as SEDAR-GRIFFIN^{N} and SEDAR-SQUIRREL^{N}). We ran the fuzzers with these seeds on MonetDB, DuckDB, and ClickHouse for 24 hours with 5 times repeated.

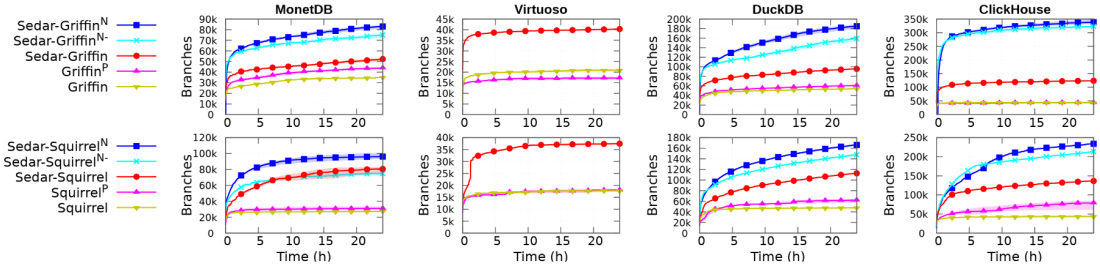


Figure 8: The branch coverage of DBMS fuzzers on MonetDB, Virtuoso, DuckDB, and ClickHouse. Displayed are the medians and the 95% confidence intervals of 5 repeated times. Be aware that Virtuoso does not have open-source native test cases.

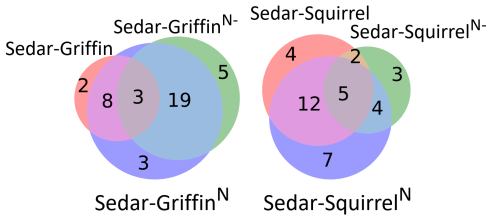
The branch coverage of these fuzzers is displayed in Figure 8. When only using the native test cases as initial seeds (i.e. SEDAR-GRIFFIN^{N-} and SEDAR-SQUIRREL^{N-}), the fuzzers achieved higher code coverage than those without native seeds by 30.61%-50.89%, and 59.14%-153.12% in DuckDB, and ClickHouse, respectively, compared to SEDAR-GRIFFIN and SEDAR-SQUIRREL. It is because these native test cases are maintained by the developer for unit testing and regression testing and have already covered a wide range of features and functionality of the DBMSs. Then, the mutability refinement helps the fuzzers to adapt these test cases, leading to the high branch coverage after fuzzing for 24 hours.

Furthermore, when fuzzing with the combination of transferred seeds and native seeds, the fuzzers SEDAR-GRIFFIN^{N} and SEDAR-SQUIRREL^{N} enhance code coverage by 10.77%-28.41%,

12.53%-16.20%, and 4.90%-9.73% in MonetDB, DuckDB, and ClickHouse, respectively, compared to SEDAR-GRIFFIN^{N-} and SEDAR-SQUIRREL^{N-}. It is because SEDAR brings various and valuable test cases from other well-tested DBMSs (i.e. SQLite, PostgreSQL, MySQL), which may have not been considered yet by the native test cases of the target DBMS. It shows that the transferred seeds can be used to augment the native seeds and achieve higher branch coverage.

Table 3: Number of bugs detected by DBMS fuzzers in 24 hours. Shown are the medians of 5 runs.

Fuzzer	MonetDB	Virtuoso	DuckDB	ClickHouse
GRIFFIN	2	2	1	0
GRIFFIN^{P}	3	0	1	0
SEDAR-GRIFFIN	10	10	2	1
SEDAR-GRIFFIN^{N-}	27	-	0	0
SEDAR-GRIFFIN^{N}	30	-	2	1
SQUIRREL	9	6	0	0
SQUIRREL^{P}	5	8	0	0
SEDAR-SQUIRREL	18	22	3	2
SEDAR-SQUIRREL^{N-}	13	-	0	1
SEDAR-SQUIRREL^{N}	23	-	3	2



(B) SEADER v.s. Native

Figure 9: Relationships between the bugs in Table 3.

The numbers of bugs in 24 hours by fuzzers with the two sets of seeds are shown in Table 3 and Figure 9 (B). SEDAR-GRIFFIN^{N} and SEDAR-SQUIRREL^{N} discovered 6 and 14 more bugs than utilizing only native seeds. These results show that SEDAR seeds can improve the branch coverage and bug detection of fuzzers on DBMSs by augmenting their native test cases with the transferred seeds of SEDAR.

Requested Change 2: Adjust the contribution claims (e.g., in abstract/intro) accordingly

Reply: We have adjusted the contribution claims according to the newly added experiment results: (1) We have added the description of the newly added experiment (the combination of native seeds and SEDAR vs. native seeds only) in the abstract and introduction. Specifically, the fuzzers with the combination of native seeds and SEDAR enhanced branch coverage by 4.90%-16.20% and 9.73%-28.41% compared to native seeds only. (2) We have rewritten the second contribution to avoid overselling: the old version's sentence "by leveraging this approach, existing DBMS fuzzers can be applied to a wider range of DBMSs, significantly expanding their scope and impact" is replaced with "these seeds can also be utilized to augment the native seeds, thereby increasing code coverage and facilitating the detection of unknown bugs." (3) We have explicitly stated in the abstract and introduction how many bugs were uniquely detected by SEDAR with its transferred seeds only. Specifically, 60 bugs out of 70 are uniquely discovered by SEDAR with the transferred seeds; the other 10 bugs can be detected by native seeds as well.

Requested Change 3: Discussing the threat to external validity of just experimenting with ChatGPT

Reply: We have adjusted the discussion on "dependencies on LLMs" in the Discussion section to focus on the external validity of just experimenting with ChatGPT. We attempted the transfer examples presented in the paper on different LLMs, with the results shown in Table 8. As

illustrated, Three LLMs including ChatGLM2-130B, CodeLlama-13B, and CodeLlama-34B successfully transferred these test cases.

Table 8: The transfer results of other LLMs on the SQL transfer cases in Figure 5 and Figure 7.

Model	Figure 5 create table	Figure 5 replace	Figure 5 select	Figure 7 create table	Figure 7 insert
Llama-2-7B	✗	✗	✗	✗	✗
Llama-2-13B	✗	✓	✓	✗	✗
Llama-2-70B	✓	✓	✓	✗	✓
ChatGLM2-6B	✗	✓	✗	✗	✗
ChatGLM2-130B	✓	✓	✓	✓	✓
CodeLlama-7B	✓	✓	✓	✗	✓
CodeLlama-13B	✓	✓	✓	✓	✓
CodeLlama-34B	✓	✓	✓	✓	✓

✓: transferred correctly; ✗: transferred wrongly.

From the table, we can preliminarily conclude that LLMs fine-tuned for code tasks or those with more parameters tend to output better results. Additionally, we mentioned that for newly developed DBMSs or new DBMS features, fine-tuning LLMs with DBMSs' documentation is a solution to improve outcomes.

Requested Change 4: Analyzing the low syntactic/semantic accuracy of the translated seeds
Reply: In the Discussion section, we have added the reasons for the low syntactic/semantic accuracy and proposed feasible solutions for improvement. There are three major reasons for the accuracy: (1) The different support of SQL features in different DBMSs. If the feature in the original statement is not supported by the target DBMS, the LLMs will make up a non-executable statement with unsupported grammars for the target DBMS. For example, the Virtuoso database primarily supports the features of graph databases, and only offers basic support for SQL features. It makes the correctness of Virtuoso lower than the other three DBMSs, since many statements do not have equivalent ones in Virtuoso. (2) The hallucinations of LLMs. As a type of probabilistic model, LLMs just predict the response with a high probability based on their training data, even if the response is factually incorrect. This means that the LLMs can sometimes produce inaccurate results of SQL transfer, leading to syntactical and semantical incorrectness. (3) The contextual dependencies between SQL statements. While the previous two reasons affect the accuracy of transferring one statement, The dependencies between SQL statements will spread the semantical incorrectness to all statements depending on it. It makes the semantical correctness ratios significantly lower than the syntactical correctness ratios on these DBMSs. We have also pointed out that the error feedback with LLMs is a solution to improve the correctness in the discussion section.

Requested Change 5: At least commenting on the diversity of the (correctly) translated constructs, if not a full-blown baseline
Reply: In Section 5.4, we added an experiment to illustrate the "correctly translated constructs". We evaluated this using the target DBMS's functions, data types, and keywords in the semantically correct statements in non-transferred seeds and transferred seeds:

Table 6: The correct syntactic structures in the semantically correct statements of S^- and S.

DBMS	Function		Data Type		Keyword	
	S^-	S	S^-	S	S^-	S
MonetDB	107	**132**	29	29	235	**244**
Virtuoso	28	**52**	16	**17**	102	**136**
DuckDB	185	**214**	34	**39**	291	**340**
ClickHouse	83	**227**	36	**61**	124	**152**
Total	403	**625**	115	**146**	752	**872**
Increment	222	-	31	-	120	-

Based on the experimental results, we can see that the transferred seeds contain 222 more functions, 31 more data types, and 120 more keywords than original ones, showing the effectiveness of SQL transfer of SEDAR.

Replies to Reviewer A

Comment 1.1: I would be interested to also understand if Sedar can improve fuzzing for a more mature DBMS, e.g. transfer test cases from MySQL to PostgreSQL, or (generalizing) whether the can be a way to figure upfront if existing seeds are adequate or not. That's beyond the scope of this work though.

Reply: SEDAR can also improve fuzzing for a mature DBMS by transferring test cases from other DBMSs. We ran a preliminary experiment on MySQL 8.1.0. We transferred the test cases of SQLite and PostgreSQL to MySQL grammar, and ran SEDAR-SQUIRREL with these test cases with 20 instances. Within 24 hours, they discovered 4 unique crashes in MySQL, which have already been confirmed by MySQL developers.

Comment 1.2: My main concern is the reliance on an LLM and the discussion section doesn't do that much to help. It's unclear if the prompts use the LLM optimally and what are the practical limitations around re-training.

Reply: We apologize for the misunderstanding; the original intent of the paragraph was to discuss "fine-tuning" rather than "retraining." We mistakenly used the terminology. Currently, most of the popular LLMs already support fine-tuning operations. The gpt-3.5-turbo model used in this paper also supports online fine-tuning. To use the LLMs optimally, a better design of the prompts would be introducing the error feedback, as discussed in the revised Discussion section.

Comment 1.3: Table 5 shows that a large fraction of transferred SQL is incorrect. Why is that? Have you tried different prompts?

Reply: There are three reasons for incorrect transferred SQL: the different support of SQL features in different DBMSs, the hallucinations of LLMs, and the contextual dependencies between SQL statements. A solution to improve the correctness is introducing the error feedback in prompts. Details can be found in the reply to **Requested Change 4**.

Replies to Reviewer B

Comment 2.1: The costs of the proposed approach (SEDAR) are not investigated…thus reporting the costs of SEDAR is very necessary. For example, SEDAR closely relies on LLMs, which could be very costly, and moreover the step of mutability refinement could be also time-consuming.

Reply: Taking ClickHouse as an example, the cost of using SEDAR to transfer test cases from SQLite, MySQL, and PostgreSQL to ClickHouse is $26.8 for the usage of gpt-3.5-turbo API. This usage is approximately equivalent to consuming 13 million tokens. The gpt-3.5-turbo API was called via our Java code, which is executed on a 12-core 12th Gen Intel(R) Core(TM) i7-12700KF @ 3600 Mhz machine with 16GiB of main memory running Windows 11. It took 56 minutes to transfer all of the test cases to ClickHouse with 100 threads of API calls. It's worth noting that the SQL transfer with LLMs only needs to be executed once, since these transferred initial seeds can be reused in further fuzzing, regardless of the fuzzers to use, the number of fuzzing instances, or the duration of fuzzing. Therefore, the costs are acceptable.

With mutability refinement enabled, the number of test cases executed by the fuzzer in 24 hours dropped from an average of 7 million to 6 million, a decrease of 14%. However, considering its improvement in the effectiveness of fuzzing, this overhead is acceptable.

Comment 2.2: Regarding the step of utilizing LLM, it is unclear about the performance of LLM in this step. That is, SEDAR utilizes LLM to transfer test cases cross DBMSs, but it is unclear about the accuracy (success rate/equivalence rate) of the transfer process.

Reply: The accuracy of a single statement of the transfer process can be evaluated through the syntactic correctness ratios. As shown in Table 5, the accuracy is 75.4%, 53.9%, 87.1%, and 79.3% in MonetDB, Virtuoso, DuckDB, and ClickHouse, respectively. The accuracy of the entire test case can be evaluated based on the semantic correctness ratios namely 37.0%, 16.0%, 35.2%, and 29.7% in the four DBMSs. Since SQL statements in a test case usually have dependencies on some of the previous statements within the test case, the accuracy of transferring a test case tends to be lower than that of individual statements. We detailedly discussed the correctness ratios and the solution to improve them in the revised Discussion Section.

Comment 2.3: The baseline using the built-in test cases in the target DBMS should be compared in the main experiment, rather than the preliminary experiment in Discussion.

Reply: We have reported the full evaluation of introducing native test cases into initial seeds in the revised Section 5.3. Details can be found in the reply to **Requested Change 1**.

Comment 2.4: The built-in test cases in DuckDB and ClickHouse with the component of mutability refinement cannot detect any bugs (significantly underperforming the test cases in other DBMSs), which is not easy to understand with regard to the effectiveness. Please provide explanation about it.
Reply: DuckDB and ClickHouse officially support many fuzzing tools, such as libFuzzer, SQLsmith, and SQLancer. Furthermore, they have developed their own fuzzing tools (e.g., DuckFuzz for DuckDB, and ASTFuzzer for ClickHouse), and have fuzzed with their native test cases for a long time (both DuckDB and ClickHouse have incorporated fuzzers into their CI). This means that many of the possible code regions and bugs explored from native seeds were already uncovered by their long-time fuzzing. Thus, repeating the fuzzing with native seeds tends to just encounter duplicates of bugs that have already been discovered and fixed. It makes it difficult to detect new unknown bugs from the native seeds within a limited duration of fuzzing, such as 24 hours in the evaluation. We also pointed out these reasons in the revised paragraph of the bug comparison.

Comment 2.5: It is also intuitive to compare with the generated seeds. Several generation approaches have been discussed in Related Work. It is important to investigate whether transferring test cases is a better seed source than generated seeds. Also, the generation source can be more efficient, which is an advantage compared to the source of transferring test cases.
Reply: The seed generation works (e.g. SLF, Tensilefuzz) primarily use symbolic execution or input field checks to explore inputs. These approaches are suitable for those relatively small libraries and programs with binary streams as inputs (e.g. libzip, libpng). However, for large software like DBMSs, which have complex code logic, vast code paths, and strict input checks, such methods do not yield effective results. These seed generation approaches will just tend to mainly explore the code regions that focus on input checks (e.g. the parsers of DBMSs), instead of the core logic of DBMSs (e.g. optimizers, storage engines, etc). This makes it hard to trigger the complex functionalities of DBMSs. Nevertheless, these approaches could be a choice to deeply test the parser components of DBMSs. Moreover, combining both the transferred seeds and those generated seeds may also enhance the effectiveness of DBMS fuzzing.

Comment 2.6: Third, the step of LLM-based transfer is the core of SEDAR, but its contribution to the fuzzing effectiveness is not investigated in the study. In Section 5.4, the fuzzing effectiveness of this step is not investigated, and it just reports the quality of the seeds in terms of several metrics. It is necessary to include a baseline that removes the LLM-based transfer component from SEDAR (keeping the step of mutability refinement). It is a more proper baseline compared to the second baseline mentioned in Section 5.3.
Reply: We apologize for the confusion. In our experiments, the seeds we collected before transfer were from SQLite, MySQL, and PostgreSQL. Since SQUIRREL and GRIFFIN support mutation for the test cases of SQLite, MySQL, and PostgreSQL, it is not necessary to enable

mutability refinement when using these non-transferred test cases. In other words, enabling mutability refinement would not alter the results.

Replies to Reviewer C

Comment 3.1: Using a target DBMSs native tests as seeds is the most obvious baseline for fuzzing, even when it requires SEDAR's mutability analysis for making the seeds fuzzer-friendly. This baseline should have been included in the analysis of RQ1 and RQ2.
Reply: We have reported the full evaluation of introducing native test cases into initial seeds in the revised Section 5.3, including the bug detection for RQ1 and the branch coverage for RQ2. Details can be found in the reply to **Requested Change 1**.

Comment 3.2: I'm wondering how many different syntactic constructs the LLM-translation successfully makes. For instance, I was struck by the observation in lines 950-951: "For ClickHouse, the improvement mainly comes from the CREATE TABLE statement" If that is the case for all translations, perhaps the LLM-based translation is overkill and could be replaced by a simple rule-based approach.
Reply: For the **"**different syntactic constructs the LLM-translation successfully makes", we evaluated it via the number of target DBMSs' functions, data types, and keywords included in the semantically correct statements. Specifically, the transferred seeds contain 222 more functions, 31 more data types, and 120 more keywords than the original ones. Details can be found in the reply to **Requested Change 5**.

Table 6: The correct syntactic structures in the semantically correct statements of S^- and S.

DBMS	Function		Data Type		Keyword	
	S^-	S	S^-	S	S^-	S
MonetDB	107	**132**	29	29	235	**244**
Virtuoso	28	**52**	16	**17**	102	**136**
DuckDB	185	**214**	34	**39**	291	**340**
ClickHouse	83	**227**	36	**61**	124	**152**
Total	403	**625**	115	**146**	752	**872**
Increment	222	-	31	-	120	-

For the CREATE TABLE statements of ClickHouse, since the SQL statements often depend on some specific tables in the database, the successful creation of tables in ClickHouse can affect the execution of other statements. However, only transferring these CREATE TABLE statements is not enough to achieve the performance of SEDAR. As demonstrated in Table 6, SEDAR also successfully transferred 144 more functions, 49 more data types, and 28 more keywords in ClickHouse. They collectively contribute to the quality of transferred seeds along with the transfer of CREATE TABLE statements. We have explained the experiment results of the correct syntactic structures of transferred seeds in Section 5.4 to avoid misunderstanding.

Comment 3.3: Another miss is that Sec 6: Discussion doesn't really do an honest and complete discussion of the limitations of SEDAR. To point out just a couple, per Table %, a significant percentage of SEDAR-translated tests are syntactically or semantically incorrect (even though SEDAR improves on the baseline). What are the main causes? This deserves discussion, preferably with concrete examples.
Reply: There are three main reasons for incorrect transferred SQL: the different support of SQL features in different DBMSs, the hallucinations of LLMs, and the contextual dependencies between SQL statements. Details can be found in the reply to **Requested Change 4****.**

Comment 3.4: Second, the discussion on "Dependency on LLMs" omits the fact that SEDAR's current implementation uses ChatGPT, which is by no means a standard component (i.e., different LLMs perform very differently). Would the SEDAR technique work with another LLM (say LLaMA or CodeT5+), particularly the translation stage or would some more creative prompt engineering and/or RAG solution be required?
Reply: We attempted the transfer examples presented in the paper on different LLMs and preliminarily concluded that LLMs fine-tuned for code tasks (e.g. CodeLlama, which is fine-tuned from Llama-2) or those with more parameters (e.g. ChatGLM-130B) tend to output better results. Details can be found in the reply to **Requested Change 3**.

Comment 3.5: The baseline of using (with mutability refinement) existing tests of the target DBMS should be included in the main evaluation and the claims regarding SEDAR (i.e., how many additional coverage and how many additional bugs it finds) adjusted on the basis of that.
Reply: We have reported the full evaluation of introducing native test cases into initial seeds in the revised Section 5.3, including bug detection and branch coverage. Details can be found in the reply to **Requested Change 1**.

For the claims regarding SEDAR, We have adjusted the contribution claims according to the newly added experiment results, including the additional coverage and additional bugs. Details can be found in the reply to **Requested Change 2**.